

RTL Design Sherpa

Binary BCH Board Validation Report Genesys 2 FPGA board evidence

October 2026

Table of Contents

1 Document Information....7
1.1 Revision History....7
1.2 Scope....8
1.3 Provenance....9
2 Scope and Claims....10
2.1 What this book is....10
2.2 What is claimed....11
2.3 What is not claimed....12
3 Images and Codec Profile....13
3.1 Image matrix....13
3.2 Codec profile....14
4 Harness Architecture....15
4.1 RTL inventory....15
4.2 Host and build inventory....17
4.3 Datapath diagram....18
4.3.1 Figure 2.1: BCH AXIS datapath and register path....18
4.3.2 Figure 2.2: BCH AXI4 memory-to-memory datapath....19
4.4 Register path, clock, and reset....20
5 Campaigns and the Oracle Argument....21
5.1 Campaign inventory....21
5.2 Determinism....22
5.3 The on-chip oracle....23

6 Testing Choices..... 24
6.1 Injector modes mapped to failure mechanisms..... 24
6.2 Post-encoder injection..... 25
6.3 UART and the safety pair..... 26
6.4 Genesys 2 vs Nexys A7..... 27
6.5 AXIS vs AXI4..... 28
6.6 Deliberately not board-tested..... 29
7 Soak Results..... 30
7.1 One million blocks on the AXIS image..... 30
7.2 Soak timeline..... 31
7.2.1 Figure 4.1: BCH soak timeline over one million blocks..... 31
7.3 What 31 blk/s means..... 32
8 Honest Limits..... 33
8.1 What the board evidence does not cover..... 33
8.2 Traceability..... 34

List of Figures

Figure 2.1: BCH AXIS datapath and register path.....18
Figure 2.2: BCH AXI4 memory-to-memory datapath.....19
Figure 4.1: BCH soak timeline over one million blocks.....31

List of Tables

Table 1: Table 0.1: Revision history....7
Table 2: Table 1.1: BCH Genesys 2 image matrix....13
Table 3: Table 2.1: RTL blocks in the BCH harness....15
Table 4: Table 2.2: Host scripts and build flow....17
Table 5: Table 3.1: BCH board campaign inventory....21
Table 6: Table 3.2: Injector modes and the physical questions they answer....24
Table 7: Table 4.1: BCH soak final counters....30
Table 8: Table 5.1: Traceability from claim to stable artifact....34

1 Document Information

1.1 Revision History

Table 0.1: Revision history.

Revision	Date	Author	Description
0.1	2026-10-05	RTL Design Sherpa	Initial board validation roll-up for the BCH Genesys 2 harness.

1.2 Scope

This book covers the BCH(4224,4120) t=8 codec as exercised on the Digilent Genesys 2 board. It describes the harness, the campaign methodology, and the soak result that produced one million decoded blocks of board evidence.

1.3 Provenance

All timing and campaign results in this book come from `stable/MANIFEST.md` and `stable/results/2026-10-05_soak/`. The board used for the runs is Digilent Genesys 2 serial `200300B818A0` (Kintex-7 `xc7k325t-2`), programmed and measured with Vivado 2025.1.

2 Scope and Claims

2.1 What this book is

This report is the board's statement about the BCH(4224,4120) t=8 codec. The Genesys 2 harness runs the same campaigns in cocotb simulation and on real silicon, so when the board passes we have evidence that simulation matches hardware at 100 MHz with real I/O, real clocks, and real register traffic.

2.2 What is claimed

The soak run demonstrates that the codec, as built into the `genesys2_axis` bitstream, corrects up to the design boundary and flags everything past it:

- 1,000,000 blocks were decoded.
- 0 blocks were silently mis-decoded.
- 74,560 blocks carried more than t=8 errors; every one was reported as uncorrectable.
- 3,119,679 bits were corrected across the run.

That is a statistical, not exhaustive, proof that the decoder's correction and uncorrectable verdicts hold under continuous random traffic on the board.

2.3 What is not claimed

This book does not replace the component DV matrix. It does not prove every possible error pattern, every smaller BCH profile, or every corner of the AXI4 flavour at million-block depth. It also does not cover erasure decoding; the BCH harness has no erasure path by design. The limits in Chapter 5 say exactly what is left outside the board evidence.

3 Images and Codec Profile

3.1 Image matrix

Two Genesys 2 bitstreams were built and kept in `stable/reports/`:

Table 1.1: BCH Genesys 2 image matrix.

Flavor	Bitstream	WNS (ns)	Failing endpoints	LUTs	Notes
AXIS	`bch_loop_genesys2_axis.bit`	+0.586	0	39,953	Streaming pipe; used for the million-block soak.
AXI4	`bch_loop_genesys2_axi4.bit`	+0.493	0	47,490	Memory-to-memory chain; refuses runs that exceed job memory.

Both images are timing-clean at 100 MHz on the `k325t-2`. The WNS values were read from `stable/reports/genesys2_axis/timing_summary.txt` and `stable/reports/genesys2_axi4/timing_summary.txt`.

3.2 Codec profile

The geometry is frozen in `build-loop/rtl/bch_loop_cfg_pkg.sv`:

- Code: BCH(4224,4120), t=8, m=13.
- Shortened from n=8191 so the codeword fits the target block size.
- Parity: 104 bits.
- Beat width: 32 bits (4 symbols per beat).

The AXIS flavour streams a full codeword in `ceil(4224/32) = 132` beats; the AXI4 flavour moves the same codeword through job memories. The host reads `PROFILE` to discover the geometry and `TOPOLOGY` to discover which datapath is loaded, so the same campaign scripts drive both images without a command-line flag.

4 Harness Architecture

4.1 RTL inventory

Table 2.1: RTL blocks in the BCH harness.

Module	File	Role
`bch_loop_genesys2_top`	`build-loop/rtl/bch_loop_genesys2_top.sv`	Genesys 2 board top: 200 MHz LVDS input, MMCM to 100 MHz, UART, LEDs.
`bch_loop_top`	`build-loop/rtl/bch_loop_top.sv`	Nexys A7-100T board top, kept as the `nexys_a7_100t` target option.
`bch_loop_harness`	`build-loop/rtl/bch_loop_harness.sv`	Board-agnostic datapath, CSR wiring, observer mux, status aggregation.
`bch_axi4_pipeline`	`build-loop/rtl/bch_axi4_pipeline.sv`	AXI4 flavour: memory-to-memory codec chain over three job memories.
`bch_loop_cfg_pkg`	`build-loop/rtl/bch_loop_cfg_pkg.sv`	One source of geometry, UART baud, build ID `0x4243_4850` (“BCHP”).
`bch_loop_regs` / `bch_loop_regs_pkg`	Generated from `build-loop/rtl/bch_loop_regs.rdl`	PeakRDL-generated CSR block.
`bridge_bch_loop_axil`	Generated fabric	1-master x 3-slave AXIL-to-APB fabric.
`uart_axil_bridge`	`projects/components/utility-ip/converters/rtl/uart_to_axil4/uart_axil_bridge.sv`	UART byte stream to AXI4-Lite master.
`apb4_to_peakrdl`	`projects/components/utility-ip/converters/rtl/apb4_to_peakrdl.sv`	APB4 slave to PeakRDL cpuif shim.
`axis4_master_injector`	`rtl/amba/shared/axis4_master_injector.sv`	LFSR data source, one packet per block, per-

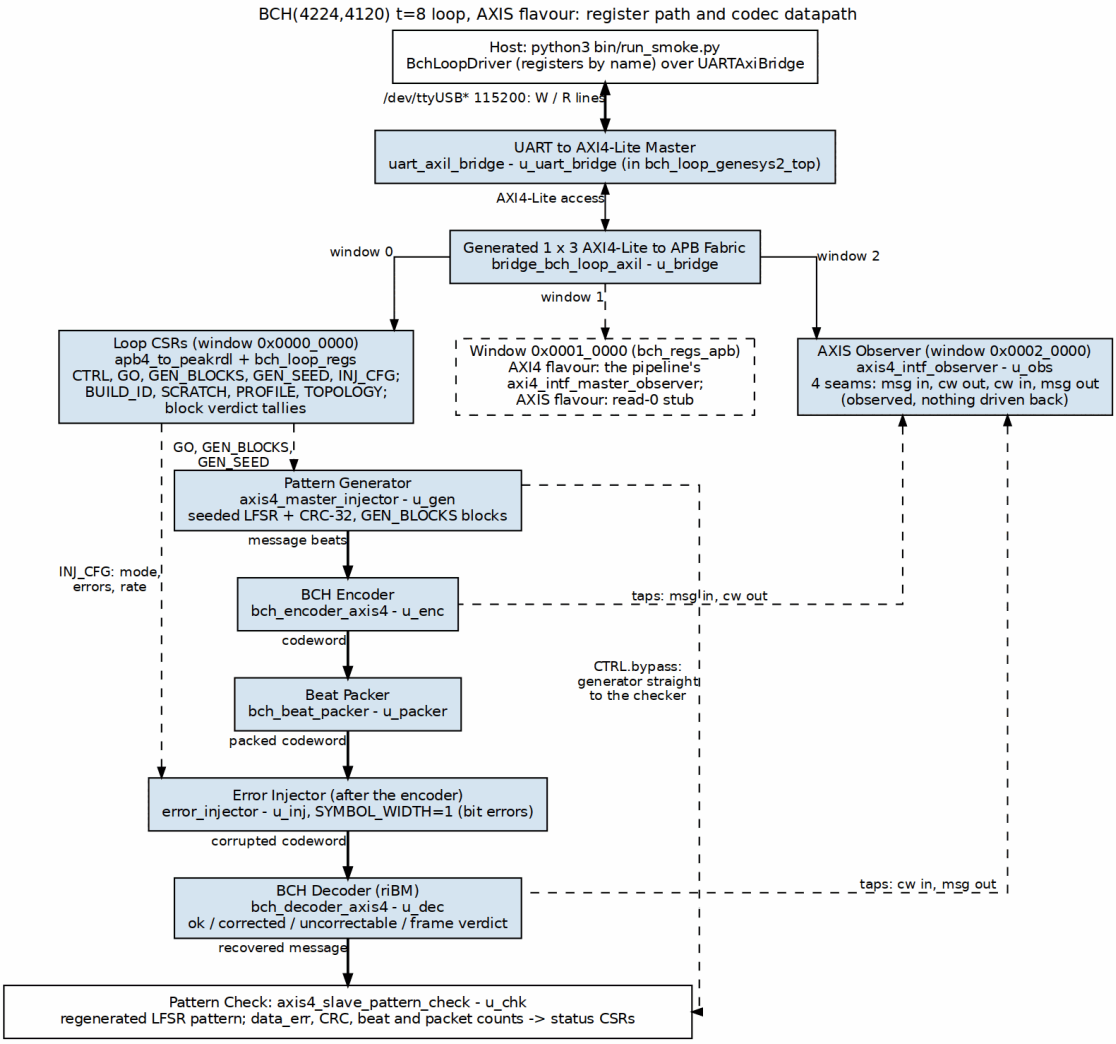
Module	File	Role
		channel CRC-32.
`bch_encoder_axis4` / `bch_encoder_core`	Component BCH RTL	BCH encoder, AXIS wrapper around the core.
`bch_beat_packer`	Component BCH RTL	Repacks encoder output into `ceil(n/BITS_PER_BEAT)` codeword beats.
`error_injector`	`projects/components/utility-ip/misc/rtl/error_injector.sv`	Shared bit-granular injector, modes 0..7, sits after the encoder.
`bch_decoder_axis4` / `bch_decoder_core`	Component BCH RTL	RIBM decoder, AXIS wrapper around the core.
`axis4_slave_pattern_check`	`rtl/amba/shared/axis4_slave_pattern_check.sv`	Independent reference checker, byte-granular CRC-32.
`axis4_intf_observer`	Shared utility RTL	APB-programmable observer on the four AXIS seams.
`axi4_intf_master_observer`	Shared utility RTL	APB-programmable observer on the AXI4 pipeline ports (AXI4 flavour only).
`axi_bus_meter`	`rtl/amba/shared/axi_bus_meter.sv`	Four windowed bandwidth meters inside the harness.

4.2 Host and build inventory

Table 2.2: Host scripts and build flow.

Script / target	File	Role
`bch_env.py`	`bin/bch_env.py`	Path anchor for shared host code.
`run_smoke.py`	`bin/run_smoke.py`	Campaign runner.
`Init` / `Smoke` / `Sweep` / `RandomCampaign` / `Soak`	`bin/seq_*.py`	Deterministic campaign sequences.
`BchLoopDriver` / `RunResult`	`build-loop/host/bch_loop.py`	By-name register access and status collection.
`bch_loop_programs`	`build-loop/host/bch_loop_programs.py`	Shared authored-once programs.
`host_bch_loop.py`	`build-loop/host/host_bch_loop.py`	CLI front-end.
`make -C build-loop bitstream`	`build-loop/Makefile`	Vivado build; set `BCH_TARGET=genesys2` and `BCH_IFACE={AXIS,AXI4}`.
`bin/build_image_matrix.sh`	`bin/build_image_matrix.sh`	Serial build of both Genesys 2 bitstreams.

4.3 Datapath diagram

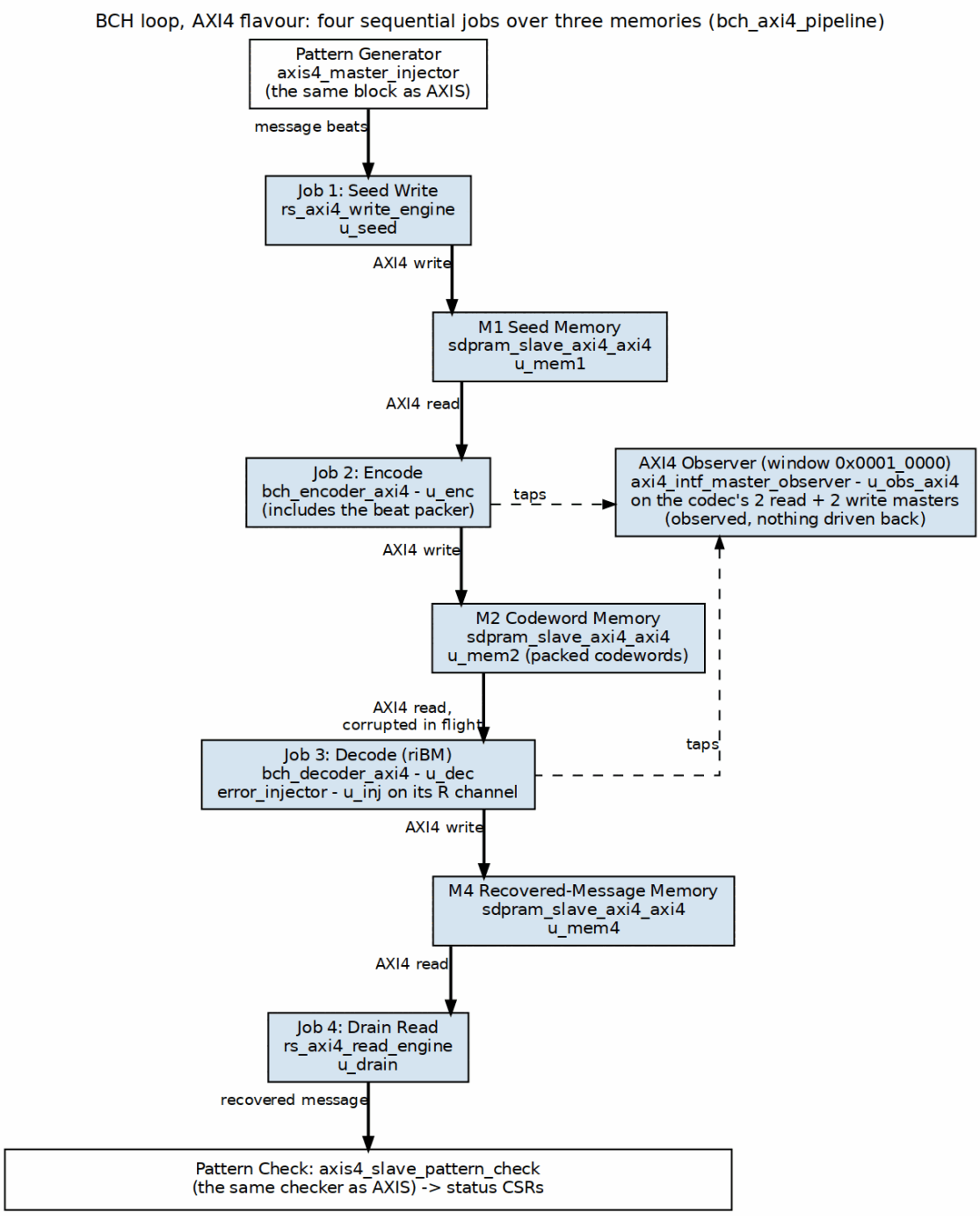


BCH AXIS datapath and register path

4.3.1 Figure 2.1: BCH AXIS datapath and register path.

Source: 01_bch_axis_loop.dot

The AXI4 flavour replaces the streaming seams with job memories; the codec itself is identical.



BCH AXI4 memory-to-memory datapath

4.3.2 Figure 2.2: BCH AXI4 memory-to-memory datapath.

Source: 02_bch_axi4_loop.dot

4.4 Register path, clock, and reset

The host talks by name, not by offset: `BchLoopDriver` wraps `UARTAxiBridge` with `UartRegisterMap` over the generated `bch_loop_regs_regmap.py`. The only raw addresses are the two expansion-window bases (`OBS_AXI4_BASE = 0x00010000` and `OBS_AXIS_BASE = 0x00020000`), which match the bridge fabric definition.

Clock and reset come from `bch_loop_genesys2_top`: the 200 MHz LVDS system clock passes through `IBUFDS` and an `MMCME2_BASE` with `CLKFBOUT_MULT_F = 6` and `CLKOUT0_DIVIDE_F = 12`, giving a 100 MHz harness clock. That frequency was chosen because the BCH loop and the UART divisor both close timing comfortably at 100 MHz on the k325t-2.

5 Campaigns and the Oracle Argument

5.1 Campaign inventory

Table 3.1: BCH board campaign inventory.

Campaign	What it proves	Determinism
`init`	BUILD_ID, SCRATCH round-trip, PROFILE, TOPOLOGY.	Deterministic.
`smoke`	Bypass, clean, e=t, e=t+1, deterministic DEBUG walk.	Deterministic.
`sweep`	Exact correction counts for e = 0 .. 2t+2.	Deterministic.
`random`	Fresh seeds, mixed injector modes, 64 runs x 16 blocks.	Host-seeded `random.Random`; reproducible on demand.
`clusters`	Spatially correlated errors model a bad DRAM row or NAND page.	Deterministic combo matrix.
`localized`	Windowed errors model a half-plane failure.	Deterministic combo matrix.
`badblock`	Most blocks clean, a hard minority very bad.	Deterministic combo matrix.
`soak`	Long-running mixed-mode stress; the million-block run.	Host-seeded; replayable.

5.2 Determinism

Every board campaign is deterministic by default: `gen_seed` and `inj_seed` start at 1, and the `seq_*.py` scripts produce bit-exact runs in cocotb simulation and on the board. A failure you can replay is a failure you can fix. `seq_random.py` still varies its draws, but those draws come from a host-side `random.Random(seed)` seeded by the caller, so the variation is reproducible when needed.

5.3 The on-chip oracle

The decoder uses riBM with odd-syndrome computation and even syndromes by squaring. It runs a second-syndrome recheck (`ENABLE_RECHECK`) and performs flip-only correction. The checker independently regenerates the expected LFSR pattern and reports `data_err` on any mismatched beat. Together they form the on-chip oracle: the decoder decides whether it thinks it corrected the block, and the checker decides whether the recovered bytes actually match the reference. That pairing is what makes a silent mis-decode detectable.

6 Testing Choices

6.1 Injector modes mapped to failure mechanisms

The shared `error_injector` exposes eight modes, and each one answers a different physical question:

Table 3.2: Injector modes and the physical questions they answer.

Mode	Failure mechanism exercised
0	None: clean run baseline.
1	Exact count: the e <= t / e > t correction boundary.
2	Burst: raw bit-error behaviour on a noisy channel.
3	Rate: continuous random bit-error rate.
4	Clusters: spatially correlated faults (bad DRAM row, disturbed NAND page).
5	Localized: windowed half-plane failure.
6	Badblock: retention tails and worn blocks.
7	Debug: deterministic walking bring-up pattern.

6.2 Post-encoder injection

The injector sits after the encoder, not inside the generator. An error in the generator data would be encoded faithfully and the code would never see it. By corrupting the codeword, the parity bits are exposed to the same errors as the data, which is what happens when a real packet is stored, transmitted, or read back from memory.

6.3 UART and the safety pair

The host link is a plain UART at 115200 baud, driven by `uart_axil_bridge`. That keeps the harness scriptable from Python without a debug-core licence. Two pieces of shared tooling guard the board:

- `projects/fpga-systems/bin/board_lock.sh` serializes programming flows by JTAG serial.
- `projects/fpga-systems/bin/fpga_board.py` refuses to program until a JTAG readback confirms the expected Genesys 2 serial `200300B818A0` is present with a device behind it.

6.4 Genesys 2 vs Nexys A7

The harness directory moved from `NexysA7/` to `Genesys2/` on 2026-10-05. `BCH_TARGET` still accepts `nexys_a7_100t` as a build option, but the Genesys 2 is the primary target. The k325t-2 has the room and timing margin to hold the full BCH(4224,4120) loop at 100 MHz with both AXIS and AXI4 flavours.

6.5 AXIS vs AXI4

Two bitstreams are built: `bch_loop_genesys2_axis.bit` and `bch_loop_genesys2_axi4.bit`. The AXIS flavour is a single streaming pipe; the AXI4 flavour is a memory-to-memory job chain across four AXI4 memories. The AXI4 flavour refuses runs whose `GEN_BLOCKS` exceed the job-memory capacity rather than letting regions wrap, so long soaks belong on the AXIS image. The host reads `TOPOLOGY` to discover which datapath is loaded.

6.6 Deliberately not board-tested

Some things are out of scope. The soak campaign ran to its default time budget and passed 1,000,000 mixed-mode blocks with zero mis-decodes, but that is a statistical sample, not every pattern. BCH has no erasure path in this harness: the injector's `out_erasure` is tied off and `cfg_mark_erasure` is held low. The board validates only the `BCH(4224,4120) t=8` profile; smaller profiles are covered in simulation and component DV.

7 Soak Results

7.1 One million blocks on the AXIS image

The 2026-10-05 soak ran the `genesys2_axis` image on board serial `200300B818A0`. Final counters from `stable/results/2026-10-05_soak/battery.json`:

Table 4.1: BCH soak final counters.

Metric	Value
blocks_final	1,000,000
wall_s	31,910
blk_per_s	31.0
clean	111,957
corrected	768,922
uncorrectable	119,121
bits_corrected	3,119,679
failing_runs	0
over_t_blocks	74,560
misdecoded	0
misdecode_rate	0.00e+00 (1 in 74,560)

The confidence statement is:

> 1,000,000 blocks, 0 mis-decodes, 74,560 over-t blocks all flagged, 3,119,679 bits corrected, 8h52m at 31 blk/s UART-bound.

7.2 Soak timeline

Fig 3: soak timeline

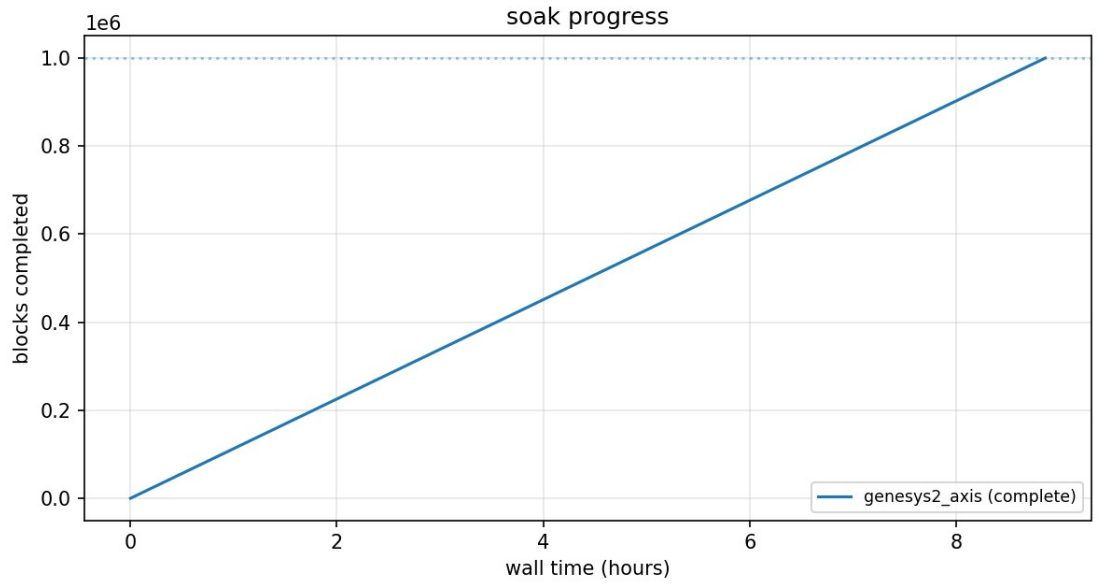


fig_3_soak_timeline.png

7.2.1 Figure 4.1: BCH soak timeline over one million blocks.

7.3 What 31 blk/s means

The codec itself can run at roughly 500k blocks per second at 100 MHz. The soak measured 31 blocks per second because the host poll loop, running over a 115200-baud UART, is the bottleneck. That is not a hardware limit; it is a fixture limit. Any future soak that wants to go deeper or faster should size the host interaction accordingly, or move to a faster host interface, rather than inferring decoder throughput from the soak rate.

8 Honest Limits

8.1 What the board evidence does not cover

- The soak is statistical, not exhaustive. One million blocks is large, but it does not visit every possible error pattern.
- The AXI4 flavour was not run to million-block depth. It refuses large runs because of job-memory capacity, so long soaks belong on AXIS.
- BCH has no erasure path in this harness by design; the injector's `out_erasure` is tied off.
- The board validates only the `BCH(4224,4120) t=8` profile. Smaller profiles like `(63,57) t=1` are covered in simulation and component DV.
- The component DV matrix exercises the codec in ways the board harness does not replicate.

8.2 Traceability

Every quantitative claim in this book can be traced to an artifact under `stable/`:

Table 5.1: Traceability from claim to stable artifact.

Claim	Artifact
Timing-clean at 100 MHz	`stable/reports/genesys2_axis/timing_summary.txt`, WNS +0.586 ns, 0 failing endpoints.
AXI4 timing-clean	`stable/reports/genesys2_axi4/timing_summary.txt`, WNS +0.493 ns, 0 failing endpoints.
One million blocks, zero mis-decodes	`stable/results/2026-10-05_soak/battery.json` and `FINDINGS.md`.
74,560 over-t blocks all flagged	`stable/results/2026-10-05_soak/battery.json`, `soak_final.over_t_blocks`.
3,119,679 bits corrected	`stable/results/2026-10-05_soak/battery.json`, `soak_final.bits_corrected`.